The Journal of Open Source Software

hdlib 2.0: extending machine learning capabilities of Vector-Symbolic Architectures

Fabio Cumbo [1¶], **Kabir Dhillon** [2], **and Daniel Blankenberg** [1,3]

1 Center for Computational Life Sciences, Cleveland Clinic Research, Cleveland Clinic, Cleveland, Ohio, United States of America **2** College of Engineering, Ohio State University, Columbus, Ohio, United States of America **3** Department of Molecular Medicine, Cleveland Clinic Lerner College of Medicine, Case Western Reserve University, Cleveland, Ohio, United States of America ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 23 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Following the initial publication of *hdlib* (Cumbo et al., 2023), a Python library for designing Vector-Symbolic Architectures (VSA), we introduce a major extension that significantly enhances its machine learning capabilities. VSA, also known as Hyperdimensional Computing, is a computing paradigm that represents and processes information using high-dimensional vectors. While the first version of *hdlib* established a foundation for manipulating these vectors, this update addresses the growing need for more advanced modeling within the VSA framework. Here, we present four extensions: significant enhancements to the existing supervised classification model also enabling feature selection, and new regression, clustering, and graph-based learning models. Furthermore, we propose the first implementation ever of Quantum Hyperdimensional Computing with quantum-powered arithmetic operations and a new Quantum Machine Learning model for supervised learning (Cumbo, Li, et al., 2025).

hdlib remains open-source and available on GitHub at https://github.com/cumbof/hdlib, and distributed through the Python Package Index (*pip install hdlib*) and Conda (*conda install -c conda-forge hdlib*).

Statement of need

The successful application of VSA across diverse scientific domains has created a demand for more sophisticated machine learning models that go beyond basic classification. Researchers now require tools to tackle regression tasks, model complex relationships in structured data like graphs, and better optimize models by identifying the most salient features.

This new version of *hdlib* directly addresses this need. While other libraries provide foundational VSA operations (Heddes et al., 2023; Kang et al., 2022; Simon et al., 2022), *hdlib* now introduces a cohesive toolkit for advanced machine learning that is, to our knowledge, unique in its integration of regression, clustering, graph encoding, and enhanced feature selection within a single framework. These additions empower researchers to move from rapid prototyping of core VSA concepts to building and evaluating complex machine learning pipelines that are now used in the context of different problems in different scientific domains (Cumbo et al., 2020; Cumbo, Truglia, et al., 2025; Cumbo, Dhillon, Joshi, Chicco, et al., 2025; Cumbo, Dhillon, Joshi, Raubenolt, et al., 2025; Cumbo, Chicco, et al., 2025; Cumbo & Chicco, 2025; Joshi et al., 2025).

Extending Machine Learning functionalities

The new architecture is summarized in Figure 1.

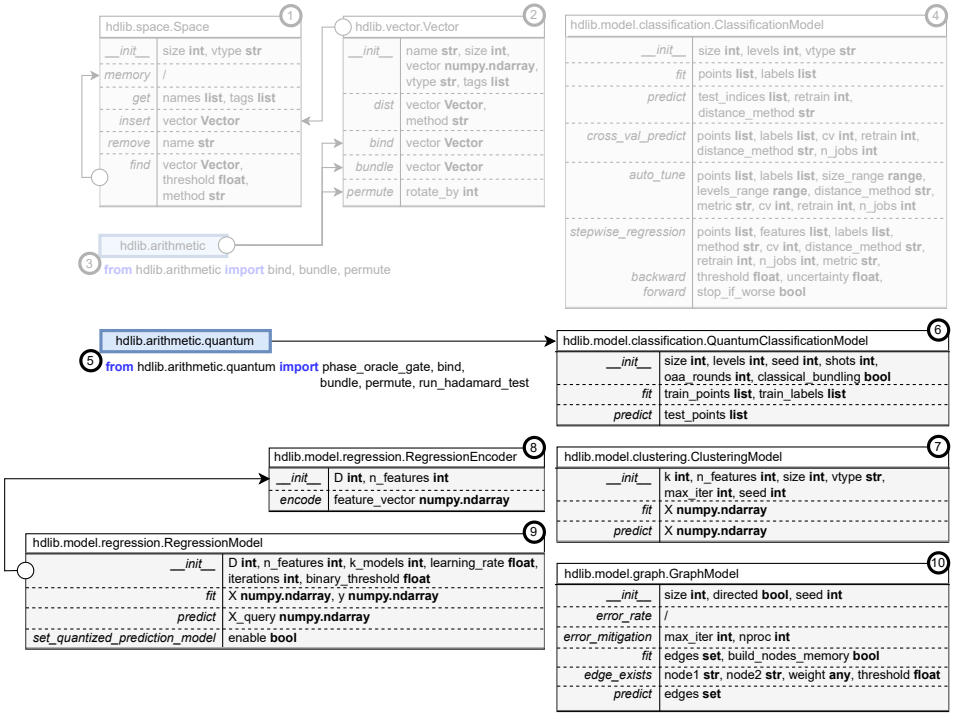

Figure 1: An overview of the *hdlib* 2.0 library architecture, highlighting the distinction between the original (top, transparent) and new components (bottom). Foundational classes from version 1.0 include `hdlib.space.Space` (Class 1), `hdlib.vector.Vector` (Class 2), `hdlib.arithmetic` module (Class 3), and the `hdlib.model.classification.ClassificationModel` (Class 4). This work introduces major new functionalities through the `hdlib.model` module comprising the new `clustering.ClusteringModel` (Class 5), `regression.RegressionEncoder` (Class 6) and `regression.RegressionModel` (Class 7), and `graph.GraphModel` (Class 8), creating a comprehensive toolkit for VSA-based machine learning.

We have significantly expanded `hdlib.model` to cover a broader spectrum of learning paradigms:

- **Classification**: the `ClassificationModel` now features advanced model optimization tools. We introduced an improved stepwise regression method for identifying feature importance and an `auto_tune` method that performs parameter sweeps to optimize vector dimensionality and level vectors;
- **Clustering**: the new `ClusteringModel` implements k-means clustering in hyperspace (Gupta et al., 2022). It iteratively refines centroids by bundling constituent data hypervectors, using cosine similarity to assign points to clusters until convergence;
- **Regression**: we implemented a `RegressionModel` and `RegressionEncoder` based on the RegHD algorithm (Hernández-Cano et al., 2021). This module maps inputs to a high-dimensional manifold and employs a multi-model strategy, where final predictions are a confidence-weighted sum of multiple regression models trained on distinct clusters of the data;
- **Graphs**: the `GraphModel` encodes directed and undirected weighted graphs into single hypervectors (Poduval et al., 2022). By compressing node neighborhoods and edge weights into a unified representation, this model enables efficient graph classification and includes an error mitigation routine to refine the graph representation.

DRAFT

Quantum Hyperdimensional Computing

We introduce Quantum Hyperdimensional Computing (QHDC), a foundational paradigm designed to run on quantum devices (Cumbo, Li, et al., 2025). The `hdlib.arithmetic.quantum` module implements the QHDC arithmetic using IBM's Qiskit framework (Javadi-Abhari et al., 2024):

- **Encoding**: we employ a phase encoding strategy where bipolar hypervectors are mapped to the relative phases of a uniform superposition state, enabling efficient algebraic manipulation;
- **Binding**: realized via quantum phase oracles that map element-wise multiplication to the sequential application of phases;
- **Bundling**: implemented as a quantum-native averaging process using a Linear Combination of Unitaries (LCU) (Chakraborty, 2024) followed by Oblivious Amplitude Amplification (OAA) (Guerreschi, 2019);
- **Permutation**: achieved using the Quantum Fourier Transform (QFT) (Weinstein et al., 2001) to induce cyclic shifts in the computational basis;
- **Similarity**: computed via the Hadamard Test to estimate the real part of the inner product (cosine similarity) between quantum states.

Finally, we provide a `QuantumClassificationModel` for supervised learning as a totally new approach to Quantum Machine Learning.

With the integration of these modules, `hdlib` 2.0 provides the scientific community with a unified and powerful framework, paving the way for the development of novel, brain-inspired solutions to a broader spectrum of machine learning problems.

References

Chakraborty, S. (2024). Implementing any linear combination of unitaries on intermediate-term quantum computers. *Quantum*, *8*, 1496. https://doi.org/10.22331/q-2024-10-10-1496

Cumbo, F., Cappelli, E., & Weitschek, E. (2020). A brain-inspired hyperdimensional computing approach for classifying massive DNA methylation data of cancer. *Algorithms*, *13*(9), 233. https://doi.org/10.3390/a13090233

Cumbo, F., & Chicco, D. (2025). Hyperdimensional computing in biomedical sciences: A brief review. *PeerJ Computer Science*, *11*, e2885. https://doi.org/10.7717/peerj-cs.2885

Cumbo, F., Chicco, D., Aygun, S., & Blankenberg, D. (2025). Designing vector-symbolic architectures for biomedical applications: Ten tips and common pitfalls. *Preprints.org*, 2025–2010. https://doi.org/10.20944/preprints202510.0117.v1

Cumbo, F., Dhillon, K., Joshi, J., Chicco, D., Aygun, S., & Blankenberg, D. (2025). A novel vector-symbolic architecture for graph encoding and its application to viral pangenome-based species classification. *bioRxiv*, 2025–2009. https://doi.org/10.1101/2025.09.08.674958

Cumbo, F., Dhillon, K., Joshi, J., Raubenolt, B., Chicco, D., Aygun, S., & Blankenberg, D. (2025). Predicting the toxicity of chemical compounds via hyperdimensional computing. *bioRxiv*, 2025–2009. https://doi.org/10.1101/2025.09.12.675894

Cumbo, F., Li, R.-H., Raubenolt, B., Joshi, J., Masum, A. K. M., Aygun, S., & Blankenberg, D. (2025). *Quantum hyperdimensional computing: A foundational paradigm for quantum neuromorphic architectures*. https://doi.org/10.48550/arXiv.2511.12664

Cumbo, F., Truglia, S., Weitschek, E., & Blankenberg, D. (2025). Feature selection with vector-symbolic architectures: A case study on microbial profiles of shotgun metagenomic

DRAFT

samples of colorectal cancer. *Briefings in Bioinformatics*, *26*(2), bbaf177. https://doi.org/10.1093/bib/bbaf177

Cumbo, F., Weitschek, E., & Blankenberg, D. (2023). Hdlib: A python library for designing vector-symbolic architectures. *Journal of Open Source Software*, *8*(89), 5704. https://doi.org/10.21105/joss.05704

Guerreschi, G. G. (2019). Repeat-until-success circuits with fixed-point oblivious amplitude amplification. *Physical Review A*, *99*(2), 022306. https://doi.org/10.1103/PhysRevA.99.022306

Gupta, S., Khaleghi, B., Salamat, S., Morris, J., Ramkumar, R., Yu, J., Tiwari, A., Kang, J., Imani, M., Aksanli, B., & others. (2022). Store-n-learn: Classification and clustering with hyperdimensional computing across flash hierarchy. *ACM Transactions on Embedded Computing Systems (TECS)*, *21*(3), 1–25. https://doi.org/10.1145/3503541

Heddes, M., Nunes, I., Vergés, P., Kleyko, D., Abraham, D., Givargis, T., Nicolau, A., & Veidenbaum, A. (2023). Torchhd: An open source python library to support research on hyperdimensional computing and vector symbolic architectures. *Journal of Machine Learning Research*, *24*(255), 1–10. http://jmlr.org/papers/v24/23-0300.html

Hernández-Cano, A., Zhuo, C., Yin, X., & Imani, M. (2021). Reghd: Robust and efficient regression in hyper-dimensional learning system. *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 7–12. https://doi.org/10.1109/DAC18074.2021.9586284

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024). *Quantum computing with qiskit*. https://doi.org/10.48550/arXiv.2405.08810

Joshi, J., Cumbo, F., & Blankenberg, D. (2025). Large-scale classification of metagenomic samples: A comparative analysis of classical machine learning techniques vs a novel brain-inspired hyperdimensional computing approach. *bioRxiv*, 2025–2007. https://doi.org/10.1101/2025.07.06.663394

Kang, J., Khaleghi, B., Rosing, T., & Kim, Y. (2022). Openhd: A gpu-powered framework for hyperdimensional computing. *IEEE Transactions on Computers*, *71*(11), 2753–2765. https://doi.org/10.1109/TC.2022.3179226

Poduval, P., Alimohamadi, H., Zakeri, A., Imani, F., Najafi, M. H., Givargis, T., & Imani, M. (2022). Graphd: Graph-based hyperdimensional memorization for brain-like cognitive learning. *Frontiers in Neuroscience*, *16*, 757125. https://doi.org/10.3389/fnins.2022.757125

Simon, W. A., Pale, U., Teijeiro, T., & Atienza, D. (2022). Hdtorch: Accelerating hyperdimensional computing with gp-gpus for design space exploration. *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 1–8. https://doi.org/10.1145/3508352.3549475

Weinstein, Y. S., Pravia, M., Fortunato, E., Lloyd, S., & Cory, D. G. (2001). Implementation of the quantum fourier transform. *Physical Review Letters*, *86*(9), 1889. https://doi.org/10.1103/PhysRevLett.86.1889

DRAFT